先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 12 – Semi-Supervised Learning

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Three periods from assumptions to calculations

Time	Main question	Classroom work
Period 1: 45 min	When are pseudo-labels useful?	Inspect assumptions; calculate accepted pseudo-labels; discuss calibration and error feedback.
Period 2: 45 min	How do views, densities and margins use unlabeled inputs?	Co-training; consistency; one EM update; S3VM loss and its limitations.
Period 3: 45 min	How can graph relations and partial constraints guide learning?	Weighted label propagation; harmonic solution; constrained clustering and fair evaluation.

Learning target

Connect every gain to an assumption and compare it with a supervised baseline.

More observations do not automatically mean more supervision

$$\mathcal{D}_L = \{(x_i, y_i)\}_{i=1}^{L}, \qquad \mathcal{D}_U = \{x_j\}_{j=1}^{U}, \qquad U \gg L.$$

- Labeled data connect input patterns with target meanings.
- Unlabeled data reveal where inputs are common and which observations are similar.
- Example: 20 labeled fault records plus 2,000 unlabeled records do not become 2,020 verified labels.
- Semi-supervised learning couples these two sources through a structural assumption or regularizer.
- The objective is better prediction for the same real-label budget, not simply using every available row.

Unsupervised groups and supervised targets answer different questions

Setting	Available target knowledge	Output meaning
Unsupervised clustering	No class labels	Groups defined by the chosen geometry
Supervised classification	Verified labels for training rows	Predictions of predefined classes
Semi-supervised classification	A few verified labels plus unlabeled inputs	The same predefined classes, using extra structural assumptions

- A cluster index is not automatically a class name.
- A pseudo-label is an inferred training target, not ground truth.
- More rows do not erase these distinctions.

Inductive and transductive tasks have different boundaries

Task	What is learned?	What must evaluation state?
Inductive	A function $f(x)$ applied to future inputs	Held-out test inputs remain outside training.
Transductive	Labels for the specified unlabeled set	The method may see those test inputs, but never their target labels.

Two applications

Deploying a fault classifier next month is inductive. Labeling one fixed archive using its entire similarity graph can be transductive.

Fair comparison

Do not compare a method using the test input graph with a strict inductive baseline without disclosing that difference.

Two prediction protocols require different held-out information

Protocol	Training may see	Training must not see
Strict inductive prediction	Training labeled rows and training unlabeled pool	Future test inputs and test labels
Declared transductive labeling	Labeled anchors and the specified prediction inputs	Target labels of those prediction inputs

- A graph over the fixed prediction archive is a transductive resource.
- A future device stream requires an out-of-sample prediction procedure.
- Compare methods under the same declared information boundary, even when neither method sees test labels.

Unlabeled data locate dense regions, not class names

One-dimensional inputs

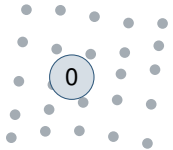
Two labeled anchors are $(x = 0, y = 0)$ and $(x = 10, y = 1)$. Unlabeled points form dense groups near 0 and 10, with few observations between 4 and 6.

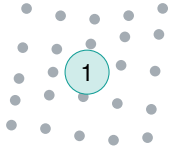
- Labeled anchors tell us the names of the two groups.
- Unlabeled observations suggest that a boundary inside the sparse gap is less disruptive than a boundary through a dense group.
- This reasoning assumes each dense group mostly belongs to one class.
- If the true label alternates within each group, the same density pattern does not reveal a useful boundary.

Main idea

Input density can guide a classifier only when density structure and label structure agree.

Unlabeled data locate dense regions, not class names





Gray observations reveal density; two labeled anchors attach class names.

The same input density can support incompatible label rules

One marginal distribution

Inputs occur in two dense intervals near 0 and 10.

- Rule A: the left interval is class 0 and the right interval is class 1.
- Rule B: within each interval, the class depends on an unobserved fine-scale pattern.
- Both rules have the same $p(x)$ but different $p(y \mid x)$.
- Unlabeled observations estimate $p(x)$ more accurately, but cannot choose between these label rules without additional evidence.
- Reliable labels and a justified coupling assumption connect the density to the classification target.

Three assumptions link input structure with labels

Assumption	Meaning	Example use
Smoothness	Nearby inputs in a high-density region tend to have similar outputs	Local consistency and graph smoothing
Cluster / low density	A dense group tends to share a class; boundaries prefer sparse regions	Cluster-and-label and S3VM
Manifold	Inputs lie near a low-dimensional structure on which labels vary smoothly	Neighborhood-graph propagation

- Similarity must be relevant to the target, not just to background or collection conditions.
- These assumptions are related; none follows from the absence of labels alone.

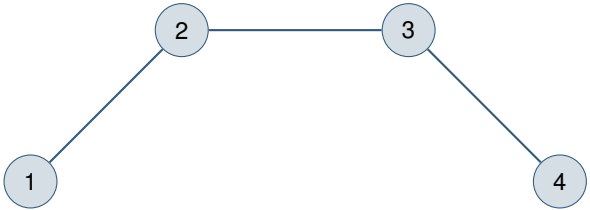
Smoothness should follow the right paths, not every short straight line

- Two points on different turns of a spiral can be close in Euclidean space but far along the data structure.
- A high-density path connecting points is more informative than a shortcut crossing a sparse gap.
- A task-relevant graph can represent such paths if its local edges are trustworthy.
- Features dominated by background or device identity can create smoothness for the wrong target.
- The assumption is about labels along meaningful input relations, not about making all nearby recorded numbers identical.

Connection to L11

Check neighborhood construction before using it to transfer labels.

Smoothness should follow the right paths, not every short straight line



Local edges define a path; shortest-path distances follow the sampled manifold.

Entropy minimization rewards certainty but not correctness

$$H(p) = -p \log p - (1-p) \log(1-p).$$

Binary prediction	Entropy in nats	Meaning
$(0.5, 0.5)$	$\log 2 \approx 0.693$	Maximum uncertainty
$(0.9, 0.1)$	approximately 0.325	More concentrated
$(1, 0)$	0	Fully certain

- Lower entropy encourages predictions away from uncertain boundaries.
- Predicting the wrong class with probability one also has zero entropy.
- True labels and suitable structural assumptions are needed to anchor this regularizer.

A shared background can make the cluster assumption fail

A failure case

Images cluster by bright versus dark backgrounds. The required classes are cats versus dogs, and both occur in both backgrounds.

- A background-based similarity graph connects many different classes.
- Cluster-and-label spreads an anchor label to the wrong animals.
- Self-training may become more confident after repeatedly learning these incorrect pseudo-labels.
- More unlabeled images strengthen the background clusters without fixing the label mismatch.
- Check class diversity inside representative neighborhoods; improve features or reduce unlabeled weight.

Conclusion

An accurate input-density model is not necessarily an accurate model of the target classes.

Out-of-domain unlabeled records can change the wrong distribution

Device A is the deployment target

The labeled set comes from A, but 80% of the unlabeled pool comes from device B with different sensor scales.

- The marginal density of the combined pool mostly describes B.
- A graph may group records by device rather than by fault state.
- A generative model may shift means toward B; pseudo-label confidence may be unreliable there.
- Check provenance, feature ranges and representative samples before choosing unlabeled influence.
- Evaluate on labeled data matching the target deployment population. More unrelated data are not automatically useful.

Start with a supervised baseline and an untouched labeled check

1. Reserve labeled validation and test sets before pseudo-labeling or graph construction choices.
2. Train a supervised model using only the labeled training budget.
3. Add the unlabeled method while keeping the same real-label budget and comparable model capacity.
4. Tune thresholds, weights and graph settings using labeled validation performance.
5. Report repeated labeled splits, class-wise scores and the supervised improvement or decline.

Small budget

With 20 training labels, one lucky split can dominate the result. Repeating the split makes uncertainty visible.

Do not tune on pseudo-label accuracy

It measures agreement with a model's own targets unless independently verified.

Keep the annotation budget visible in the experiment

You can afford 100 verified labels

Use 40 for training, 30 for validation and 30 for final testing in this illustrative split. A separate 5,000-row pool is unlabeled.

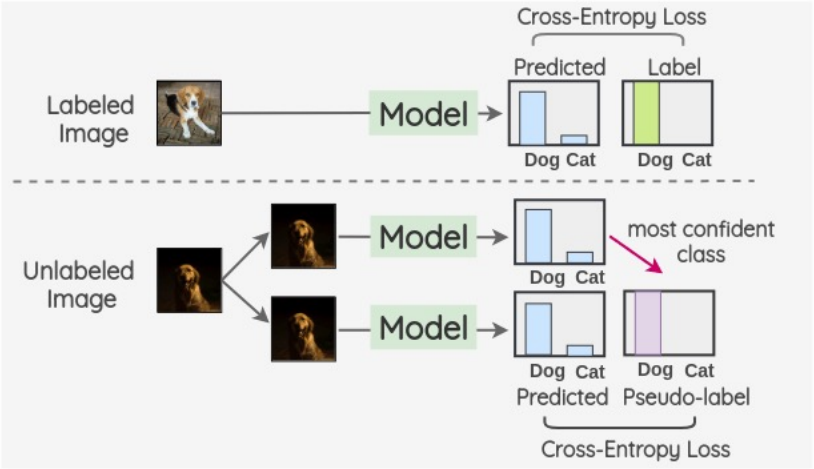
- Both baseline and semi-supervised model use the same 40 training labels.
- Validation selects thresholds or graph parameters; those 30 labels are not pseudo-label targets during this comparison.
- The final 30 test labels are used only after choices are fixed.
- Repeating smaller label subsets requires keeping evaluation labels separate.
- State whether the reported label cost counts only training annotation or the entire evaluation setup.

Self-training turns selected predictions into training targets

1. Train f_θ on the labeled training data.
2. Predict probabilities $p_\theta(y \mid x)$ on unlabeled candidates.
3. Set $\hat{y} = \arg\max_y p_\theta(y \mid x)$ and confidence $c = \max_y p_\theta(y \mid x)$.
4. Accept selected pairs $(x, \hat{y})$, for example when $c \geq \tau$.
5. Retrain with true labels plus accepted pseudo-labels; repeat while validation supports a benefit.

- A pseudo-label is a model prediction used as a target. It is not a verified label.
- Keep original labels fixed and record which targets were inferred.

Self-training turns selected predictions into training targets



Worked self-training: accept two candidates, not all four

Candidate	$p(y = 0 \mid x)$	$p(y = 1 \mid x)$	Pseudo-label	Accept at $\tau = 0.90$?
A	0.97	0.03	0	Yes
B	0.58	0.42	0	No
C	0.08	0.92	1	Yes
D	0.25	0.75	1	No

- Add $(A, 0)$ and $(C, 1)$; leave B and D unlabeled for now.
- Rejected does not mean useless or incorrect; it means evidence is insufficient under the current rule.
- Thresholding reduces exposure to uncertain targets but does not guarantee the accepted targets are correct.
- Re-evaluate candidates after the model changes.

Changing the threshold changes coverage and risk

Threshold	Accepted candidates	Inferred targets
0.95	A only	A: class 0 at confidence 0.97
0.90	A and C	A: 0; C: 1 at confidence 0.92
0.75	A, C and D	D: class 1 at confidence 0.75

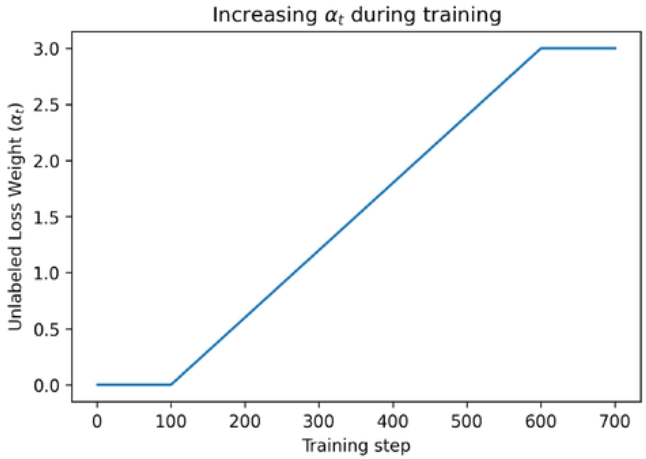
- The original candidate B has confidence 0.58 and remains excluded at all these thresholds.
- Lower thresholds increase coverage but may include less reliable targets.
- No threshold guarantees accuracy without calibration and target-domain validation.
- Count accepted rows and verify class-wise outcomes separately.

Give pseudo-labels controlled influence in the loss

$$\mathcal{L} = \frac{1}{L} \sum_{i \in L} \mathrm{CE}(y_i, p_\theta(x_i)) + \lambda \frac{1}{|A|} \sum_{j \in A} a_j \mathrm{CE}(\hat{y}_j, p_\theta(x_j)).$$

- A: accepted pseudo-labeled samples; skip the second term if none are accepted.
- a_j: optional reliability weight; λ: overall unlabeled influence.
- Averaging each source avoids letting a huge unlabeled pool dominate merely by its size.
- For pseudo-label 1 and predicted probability 0.8, the loss is $-\log 0.8 \approx 0.223$.
- If that target is wrong, reducing the loss reinforces the wrong class.

Give pseudo-labels controlled influence in the loss



Average the two data sources before choosing their relative weight

40 true labels and 4,000 accepted targets

Suppose each source has mean cross-entropy 0.2.

- Raw summed losses are $40(0.2) = 8$ and $4,000(0.2) = 800$.
- With separately averaged losses and $\lambda = 0.5$, their scalar contributions are 0.2 and 0.1.
- Averaging prevents pool size alone from setting a one-hundred-fold loss weight.
- Equal scalar contributions do not imply equal gradient directions or equal correctness.
- Reliability weights and a schedule may further control targets, but validation still decides whether they help.

Confidence needs calibration, especially with few labels

A labeled validation bin

Among 100 predictions with confidence near 0.95, only 70 are correct. The model is overconfident in that bin.

- Confidence is a score from the model; calibration compares that score with observed correctness.
- High confidence may still occur on shifted inputs or an underrepresented class.
- Use class-wise validation bins or reliability plots when the label budget supports them.
- Thresholds should be selected on real labeled validation data, not on the model's own pseudo-labels.
- With very few validation labels, report uncertainty rather than treating a precise-looking threshold as proven.

Calibration compares confidence bins with verified outcomes

Confidence bin	Verified rows	Correct rows	Accuracy
Near 0.95	100	70	70%
Near 0.80	50	40	80%

- The first bin is overconfident; the second matches its average claim in this example.
- Raising the cutoff from 0.8 to 0.9 may discard the calibrated bin while retaining the overconfident one.
- A small bin gives a noisy estimate; inspect class and domain coverage.
- Calibration is evidence from true outcomes, not agreement with pseudo-labels.

A common threshold can amplify class imbalance

Accepted pseudo-label counts

A model accepts 900 majority-class predictions and only 20 minority-class predictions at the same threshold.

- Retraining mainly on accepted majority targets can make minority examples even harder to accept next round.
- Inspect acceptance count and labeled validation performance for each class.
- Consider class-aware thresholds, quotas or weights only when their assumptions fit the task.
- Do not force a 50:50 ratio if the real class prevalence is strongly unequal.
- Maintain verified minority labels and inspect high-confidence errors from that class.

Balance is not correctness

A balanced set can still contain incorrect targets.

Overall accuracy can hide a rare-class failure

A labeled validation set

There are 100 majority examples and 10 rare-fault examples. The model correctly identifies 90 majority examples but only one rare fault.

$$\text{Accuracy} = \frac{90 + 1}{110} \approx 82.7\%, \qquad \text{MacroRecall} = \frac{0.90 + 0.10}{2} = 50\%.$$

- Rare-fault recall is only 10%, despite a seemingly acceptable overall score.
- A pseudo-label pool dominated by the majority can further reinforce this imbalance.
- Report per-class recall and costs relevant to deployment.
- Balance interventions should not replace checking target correctness or real prevalence.

Stop the loop when validation reveals error reinforcement

1. An early classifier gives an incorrect high-confidence label to a region.
2. Those pseudo-labels become extra training targets.
3. The model shifts toward the incorrect targets and becomes more confident there.
4. A larger accepted set reproduces the same mistake in the next round.

- Keep a supervised checkpoint and choose iterations using labeled validation.
- Limit accepted batches, weight uncertain targets cautiously and inspect disagreements.
- Pseudo-labels may be refreshed or removed; original verified labels remain fixed.
- Convergence of predictions is not a proof that their labels are correct.

Refreshing pseudo-labels allows an earlier inferred target to be withdrawn

Iteration	Prediction for candidate E	Action at threshold 0.9
1	Class 0 with confidence 0.94	Accept inferred class 0
2	Class 0 with confidence 0.62	Withdraw under a refresh policy
3	Class 1 with confidence 0.91	Reconsider inferred class 1

- This policy treats inferred targets as revisable, while verified labels stay fixed.
- Refreshing can limit some feedback errors; it does not prove the latest target correct.
- State whether targets are permanent, refreshed or confidence-weighted when comparing implementations.

Consistency adds another source of structure

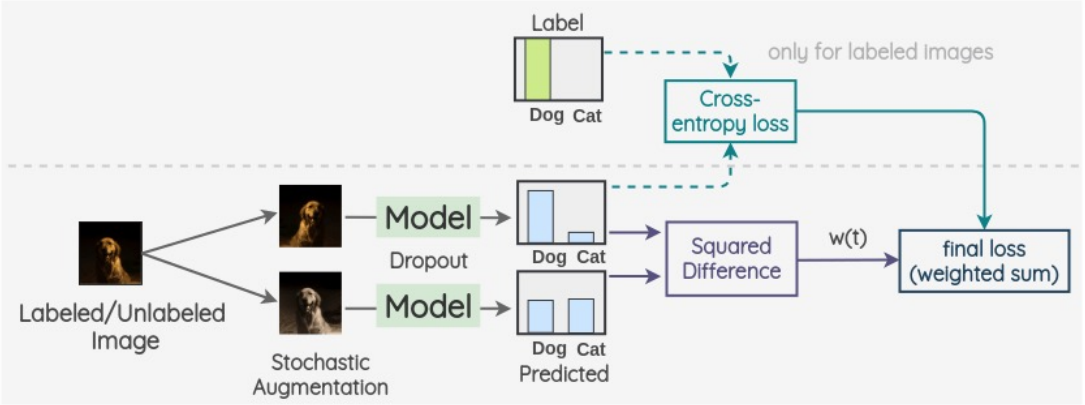
Same object, valid perturbations

A digit image shifted slightly should usually keep its label. A feature record with small measurement noise may still represent the same state.

$$\mathcal{L}_{\text{cons}} = \|p_\theta(y \mid a_1(x)) - p_\theta(y \mid a_2(x))\|^2.$$

- a_1, a_2 are transformations believed to preserve the task label.
- Unlabeled observations can supply many such consistency pairs.
- Consistent predictions can still be wrong; real labels anchor the class meanings.
- An invalid transformation can change the label, such as rotating a 6 into a 9.

Consistency adds another source of structure



An augmentation is useful only if it preserves the task label

Task	Potentially valid change	Potentially invalid change
Digit identity	Small translation without clipping	Rotation converting 6 into 9
Fault waveform class	Small sensor noise within tolerance	Deleting the short fault impulse
Sentiment label	A meaning-preserving paraphrase	Removing a negation word

- Visual similarity or small numeric change does not universally imply an unchanged label.
- Consistency loss encodes domain knowledge about valid transformations.
- Inspect representative transformed pairs before training with this assumption.

Exercise: decide which pseudo-labels deserve influence

A small-label problem

A two-class model predicts A: $(0.96, 0.04)$, B: $(0.55, 0.45)$, C: $(0.10, 0.90)$. Its 0.9-confidence validation bin has 75% correctness.

1. With threshold 0.90, which candidates are accepted and with what targets?
2. Does that threshold imply at least 90% of accepted targets are correct?
3. What happens if 95% of accepted candidates belong to the same class?
4. Name a labeled check that can stop error reinforcement.

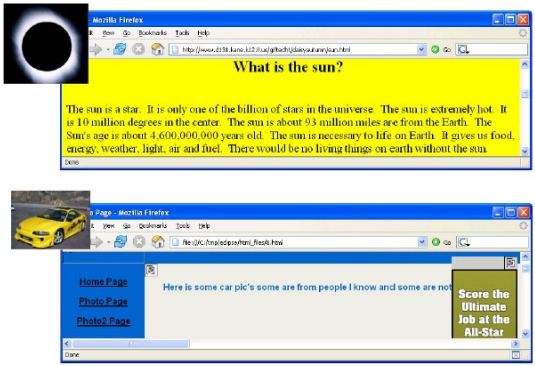
Discuss

A gets class 0; C gets class 1; B remains unlabeled. Calibration, class balance and independent validation still matter.

Solution workshop: the accepted set is only the beginning

- At threshold 0.90, A $(0.96, 0.04)$ is assigned target 0 and C $(0.10, 0.90)$ target 1; B remains unlabeled.
- The verified 0.9-confidence bin has only 75% correctness, so the cutoff does not guarantee 90% accuracy.
- If 95% of accepted rows are one class, inspect class-wise performance before imposing a balancing rule.
- Stop or roll back when independent validation reveals error reinforcement.
- A rejected hard target can still contribute a justified consistency relation, but the two losses make different assumptions.

Co-training uses two views of the same object



- One example may have image features $x^{(1)}$ and page-text features $x^{(2)}$.
- Train one classifier per view using the same labeled examples.
- Confident predictions from view 1 provide targets for view 2, and vice versa.
- The gain depends on complementary evidence, not merely having two copies of the same model.

Connection

Self-training teaches itself; co-training exchanges selected targets between views.

A natural view split should match the same target object

- A web page has image features and text features paired by the same page identifier.
- The image can show an object while the text describes that object; both can support the same target.
- A page containing unrelated captions or multiple objects can violate this pairing assumption.
- If only text is available at deployment, train and evaluate a text-view predictor rather than assuming image evidence always exists.
- Arbitrarily splitting 100 correlated sensor columns into two halves does not create two independent information sources.

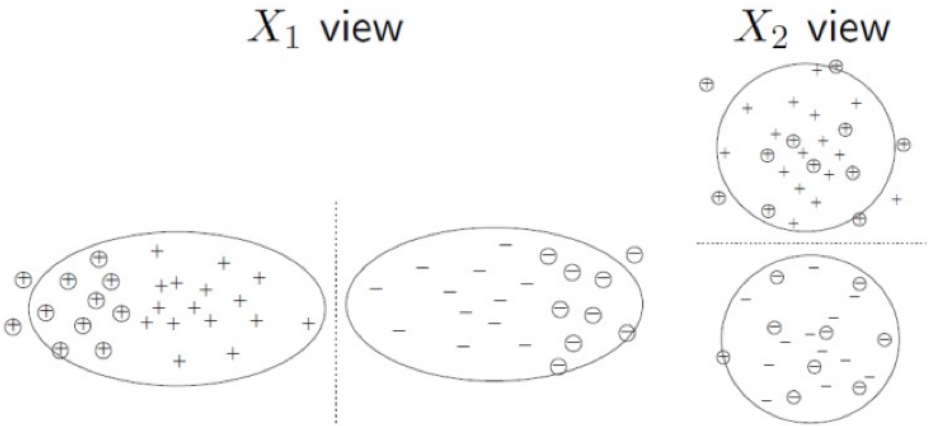
Complementary views need enough information and different errors

- **Sufficiency:** each view contains enough information to predict the class reasonably well.
- **Conditional independence** in the classical model: $p(x^{(1)}, x^{(2)} \mid y) = p(x^{(1)} \mid y)p(x^{(2)} \mid y)$.
- **Practical diversity:** one model should supply information the other has not already captured.
- Two views can be strongly correlated through a shared nuisance, such as a watermark or collection site.
- Use real labels to inspect per-view errors and exchange only targets that improve validation behavior.

Distinction

Practical co-training variants may relax the classical assumptions; theoretical guarantees do not transfer automatically.

Complementary views need enough information and different errors



Two models can agree because they share the same mistake

100 labeled checks

Image and text models each make 20 errors. In case A, all 20 errors overlap; in case B, only five overlap.

- In case A, agreement supplies little new corrective evidence on those shared errors.
- In case B, one view can sometimes be correct where the other is wrong, creating possible complementary information.
- Error overlap measured on true labels is evidence about diversity, not a complete theoretical guarantee.
- High-confidence exchange must still account for calibration, view sufficiency and sample pairing.
- Two accurate-looking copies are not automatically better teachers than one.

Worked co-training: the stronger view teaches the uncertain view

Sample	Image view p(cat)	Text view p(cat)	Action at threshold 0.9
A	0.97	0.60	Image proposes cat to text model
B	0.55	0.04	Text proposes dog to image model
C	0.93	0.06	Confident disagreement: inspect before exchanging targets

- Exchange the target for the same sample, while training each model on its own view.
- A high-confidence disagreement can signal an invalid view assumption, poor calibration or bad pairing.
- Combine views for final prediction only according to the chosen training and evaluation scheme.

Exchange a target to the other view without changing the instance

Candidate	Reliable proposer	Recipient training pair
A	Image predicts cat at 0.97	Text features of A with inferred cat target
B	Text predicts dog at 0.96	Image features of B with inferred dog target
C	Views strongly disagree	No automatic exchange; inspect evidence

- A target belongs to the object, but the recipient model uses its own feature view.
- Copying the proposer's feature vector into the other model changes the intended method.
- Keep inferred targets distinct from verified labels and monitor class-wise acceptance.

Co-regularization encourages agreement without hard exchange

$$\mathcal{L} = \mathcal{L}_L^{(1)} + \mathcal{L}_L^{(2)} + \lambda \frac{1}{U} \sum_{x \in U} \|p_1(y \mid x^{(1)}) - p_2(y \mid x^{(2)})\|^2.$$

- True labels anchor both views; unlabeled observations penalize inconsistent predictions.
- Agreement narrows the set of candidate classifiers that fit the small labeled sample.
- Two models that always predict the same class can agree perfectly and still fail.
- Model diversity, reliable features and labeled validation remain necessary.

Interpretation

Agreement is a structural constraint, not a substitute for knowing which class an observation belongs to.

Agreement loss alone has an uninformative solution

$$p_1(y \mid x) = p_2(y \mid x) = (0.5, 0.5) \quad \Rightarrow \quad \|p_1 - p_2\|^2 = 0.$$

- Both views can minimize disagreement without classifying anything confidently.
- They can also agree on the same wrong class, such as $(1, 0)$ for every input.
- Verified-label losses anchor meanings and discourage these trivial alternatives.
- Entropy or confidence terms can add concentration, but still cannot alone prove correctness.
- A combined objective should explain which term provides class information and which term provides structural regularity.

Mean Teacher smooths parameters, not verified labels

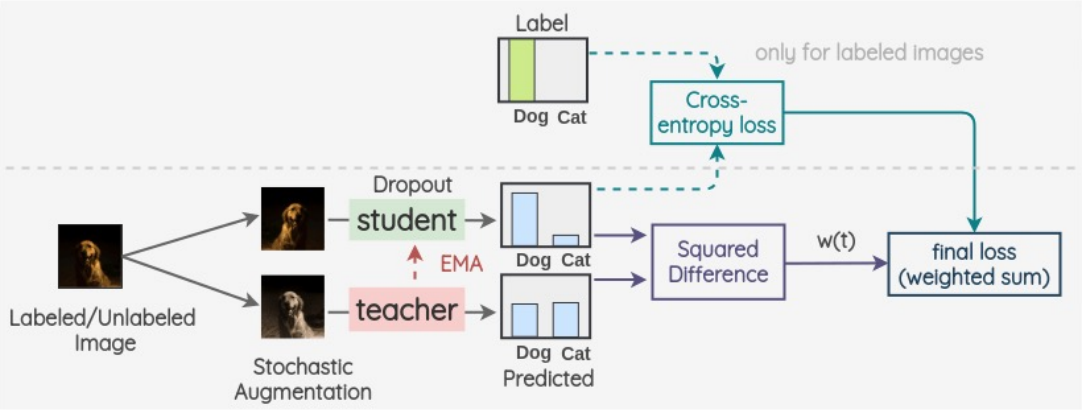
$$\theta_T \leftarrow \beta\theta_T + (1-\beta)\theta_S.$$

- The student is optimized with labeled loss plus consistency to a teacher prediction on a suitable view.
- The teacher parameters are an exponential moving average of student parameters.
- Averaging stabilizes targets over time; it does not make the teacher an external source of truth.
- For scalar parameters $\theta_T = 2$, $\theta_S = 4$, $\beta = 0.9$, the updated teacher parameter is 2.2.
- Valid augmentations and an appropriate unlabeled weight are still required.

Distinguish

Temporal ensembling averages predictions; Mean Teacher averages model parameters.

Mean Teacher smooths parameters, not verified labels



Compute two Mean Teacher updates and interpret their lag

$$\theta_T^{(1)} = 0.9(2) + 0.1(4) = 2.2, \qquad \theta_T^{(2)} = 0.9(2.2) + 0.1(5) = 2.48.$$

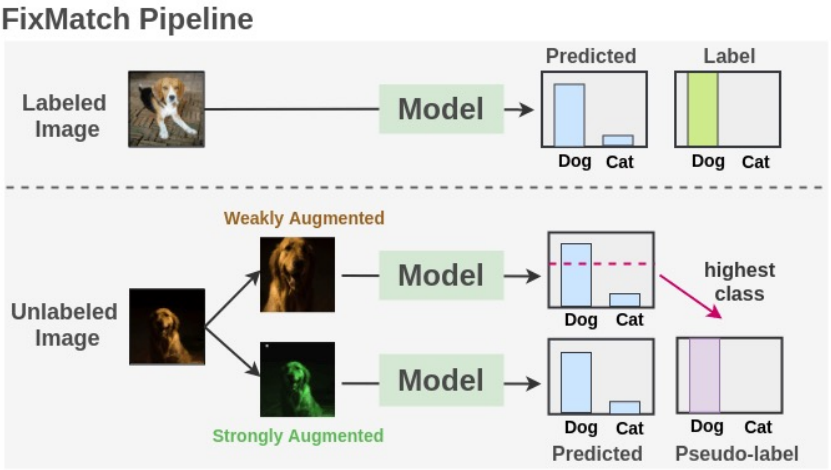
- The teacher follows recent student parameters gradually rather than copying each value immediately.
- Larger averaging coefficient means more inertia; coefficient one would freeze the teacher.
- This scalar calculation illustrates parameter averaging, not a claim that neural predictions average linearly.
- Teacher targets remain model-derived and require valid augmentations plus true-label anchors.
- Compare stable validation behavior rather than assuming a slower target is always more accurate.

FixMatch combines confidence with weak and strong views

$$q = p_\theta(y \mid a_w(x)), \quad \hat{y} = \arg\max q, \quad \mathcal{L}_U = \mathbf{1}[\max q \geq \tau]\,\mathrm{CE}(\hat{y}, p_\theta(y \mid a_s(x))).$$

- A weak augmentation provides a confidence-filtered pseudo-label.
- A stronger valid augmentation is trained to predict that same label.
- The pseudo-label target is treated as fixed during that update.
- Weak prediction $(0.96, 0.04)$ passes $\tau = 0.95$; strong prediction $(0.80, 0.20)$ receives class-0 loss $-\log 0.80$.
- The method can fail when augmentation changes the class or the weak prediction is confidently wrong.

FixMatch combines confidence with weak and strong views



A confident weak-view target controls one strong-view update

Threshold $\tau = 0.95$

Weak probabilities are $(0.96, 0.04)$; strong probabilities are $(0.80, 0.20)$.

$$\hat{y} = 0, \qquad \mathcal{L}_U = -\log 0.80 \approx 0.223.$$

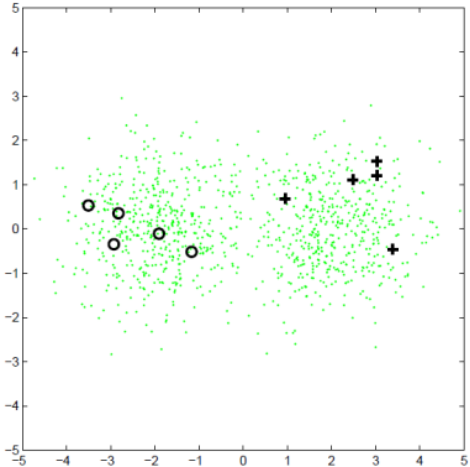
- The target is derived from the weak view and treated as fixed during this update.
- A weak confidence of 0.93 would mask this hard-target loss to zero.
- The strong view is trained to recover the same label despite a valid stronger perturbation.
- If the perturbation changes the true class, enforcing consistency is wrong.
- Neither passing the threshold nor reducing this loss verifies the inferred target.

A generative model links class names to input density

$$p(x, y = k \mid \theta) = \pi_k \mathcal{N}(x; \mu_k, \Sigma_k), \qquad p(x \mid \theta) = \sum_k p(x, y = k \mid \theta).$$

- Labeled data identify which class generated an observation.
- Unlabeled data have a missing class variable; they still inform means, covariances and mixing proportions.
- A class-conditional model connects the marginal input distribution with the prediction target.
- One Gaussian per class is an assumption, not a fact about all datasets.
- More mixture components may be needed within a class; component identity and class identity are not always the same.

A generative model links class names to input density



Labeled anchors prevent mixture components from having arbitrary class names

- An unlabeled two-component Gaussian mixture can exchange component names without changing $p(x)$.
- A verified class-0 anchor links a density component to the target name 0.
- A class can require several mixture components; component identity need not equal class identity.
- If two classes share almost the same input distribution, the marginal density gives weak information about their separation.
- Semi-supervised likelihood depends on the chosen connection between class variables and input generation.

Meaning

A good density fit needs labeled evidence before it becomes a good named classifier.

The likelihood treats unlabeled classes as missing variables

$$\ell(\theta) = \sum_{i \in L} \log p(x_i, y_i \mid \theta) + \lambda \sum_{j \in U} \log \sum_{k} p(x_j, y = k \mid \theta).$$

- A labeled sample contributes its known class likelihood.
- An unlabeled sample sums over possible classes before taking the logarithm.
- EM alternates soft class responsibilities and weighted parameter updates.
- λ can reduce unlabeled influence when the generative model is imperfect; it is not a guarantee of safety.
- Multiple initializations help with local optima, but not with a wrong model family.

Marginal likelihood sums possible classes before taking the logarithm

One unlabeled observation

Joint contributions are $p(x, y = 0) = 0.3$ and $p(x, y = 1) = 0.2$.

$$p(x) = 0.3 + 0.2 = 0.5, \quad \log p(x) = \log 0.5 \approx -0.693.$$

- Selecting only class 0 would instead use $\log 0.3 \approx -1.204$.
- The normalized class responsibilities are $(0.3/0.5, 0.2/0.5) = (0.6, 0.4)$.
- Summing and taking a maximum represent different objectives.
- A labeled class-0 observation uses its known joint contribution, not a sum over unknown classes.
- Keep density values, posterior responsibilities and verified class labels distinct.

Worked EM: calculate a soft responsibility

Two equally likely Gaussian classes

Means are $\mu_0 = 0$, $\mu_1 = 4$; both variances are 1; priors are 0.5. The unlabeled point is $x = 1$.

$$r_0 = \frac{e^{-(1-0)^2/2}}{e^{-(1-0)^2/2} + e^{-(1-4)^2/2}} = \frac{e^{-0.5}}{e^{-0.5} + e^{-4.5}} \approx 0.982.$$

$$r_1 = 1 - r_0 \approx 0.018.$$

- Equal priors and Gaussian normalizers cancel in this example.
- The point mostly contributes to class 0, but retains a small contribution to class 1.
- For a verified class-0 point, its responsibility is fixed at $(1, 0)$.

Class priors change responsibilities even at equal distances

Equal Gaussian likelihoods

A point is equally far from two means, both classes have the same variance, but priors are $\pi_0 = 0.8$ and $\pi_1 = 0.2$.

$$r_0 = \frac{0.8g}{0.8g + 0.2g} = 0.8, \qquad r_1 = 0.2.$$

- Equal distance does not imply equal posterior probability when priors differ.
- The shared positive likelihood value g cancels in this example.
- Labeled sampling may not match population prevalence; prior estimation requires care.
- A fixed balanced-label training set should not automatically be treated as a 50:50 target population.
- State whether priors are fixed, estimated or regularized.

Worked EM: update the class means with weighted samples

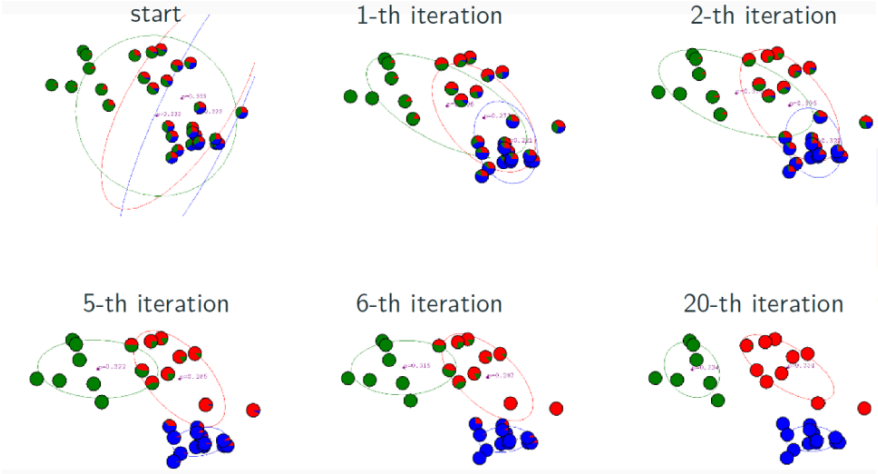
Fix variances and priors for this teaching update

One verified class-0 point is 0, one verified class-1 point is 4, and the unlabeled point 1 has responsibilities $(0.982, 0.018)$.

$$\mu_0^{\text{new}} = \frac{0 + 0.982 \times 1}{1 + 0.982} \approx 0.495, \qquad \mu_1^{\text{new}} = \frac{4 + 0.018 \times 1}{1 + 0.018} \approx 3.947.$$

- The unlabeled point pulls class 0 much more strongly than class 1.
- Recompute responsibilities using the updated means, then repeat.
- A full Gaussian EM also updates covariance and prior terms unless they are deliberately fixed.

Worked EM: update the class means with weighted samples



A covariance update uses weighted residuals around the updated mean

One class in a teaching M-step

Verified scalar values are 0 and 2. Unlabeled value 1 has responsibility 0.8 for this class.

$$\mu^{\text{new}} = \frac{0 + 2 + 0.8(1)}{2 + 0.8} = 1, \qquad (\sigma^2)^{\text{new}} = \frac{(0-1)^2 + (2-1)^2 + 0.8(1-1)^2}{2.8} \approx 0.714.$$

- The fractional count is 2.8, not three verified observations.
- The new central point adds weight without adding squared residual here.
- Covariance regularization is needed when data are sparse or a component approaches zero variance.
- This illustrates one class update with stated responsibilities, not a complete fitted two-class model.

A better density fit can produce a worse classifier

- If one class is multimodal but modeled by one Gaussian, unlabeled data can pull that Gaussian toward the wrong center.
- A very large unlabeled pool can dominate the likelihood even when class labels disagree with the density model.
- Component labels may be ambiguous without enough labeled anchors.
- Use richer class-conditional models, covariance regularization or a reduced unlabeled weight when justified.
- Compare held-out class prediction with the supervised generative model and a discriminative baseline.

Separate the questions

Likelihood improvement checks the fitted density; labeled validation checks the classification task.

A class with two modes challenges a one-Gaussian-per-class model

Class 0 has two operating modes

Verified class-0 values are near -3 and $+3$, while class-1 values are near 0.

- A single class-0 Gaussian may place its mean near 0, where another class is concentrated.
- Extra unlabeled class-0 modes can strengthen the mismatch instead of separating the classes.
- Multiple components within class 0 can represent its two modes without renaming them as two different classes.
- Compare class prediction on real validation labels and inspect whether the density family fits the observations.
- Better optimization cannot by itself make an inadequate model family correct.

Recall the supervised hinge loss before adding unlabeled points

$$f(x) = w^\top x + b, \qquad y \in \{-1, +1\}, \qquad \ell_L(y, f) = \max(0, 1 - yf).$$

True label y	Score $f(x)$	Margin $yf(x)$	Hinge loss
$+1$	1.4	1.4	0
$+1$	0.3	0.3	0.7
$+1$	-0.4	-0.4	1.4

- A correct sign is not enough for zero loss; the functional margin must reach one.
- The regularizer $\|w\|^2/2$ controls margin geometry.
- This is binary classification, not support-vector regression.

A negative true label reverses which score is favorable

$$\ell_L(y, f) = \max(0, 1 - yf), \qquad y = -1.$$

Score f	Signed margin yf	Hinge loss
-1.4	1.4	0
-0.3	0.3	0.7
$+0.4$	-0.4	1.4

- The sign of the known label determines the correct side.
- A negative score is good for this class, not inherently a classification error.
- S3VM's unlabeled term removes this known side choice by using $|f|$, which creates both its flexibility and ambiguity.

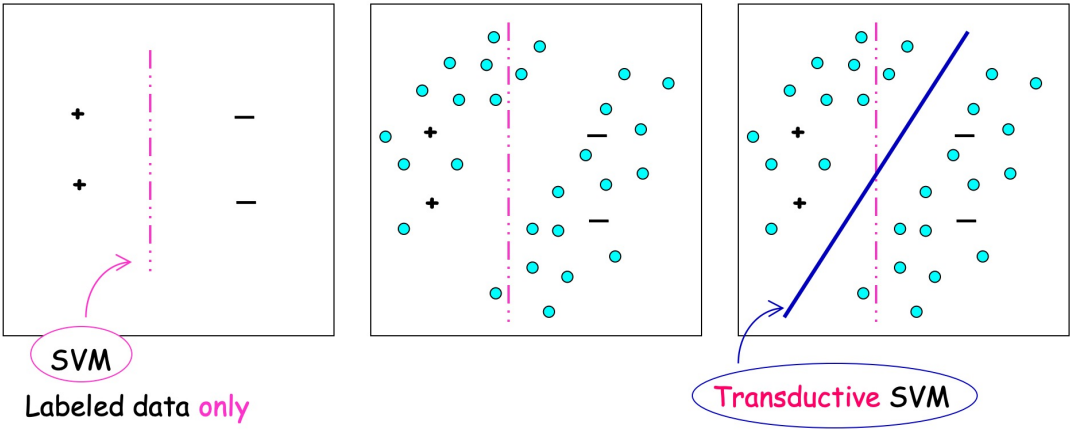
S3VM encourages a low-density classification boundary

$$\min_{y \in \{-1,+1\}} \max(0, 1 - yf) = \max(0, 1 - |f|).$$

$$\mathcal{J} = \frac{1}{2}\|w\|^2 + C_L \sum_{i \in L} \max(0, 1 - y_i f_i) + C_U \sum_{j \in U} \max(0, 1 - |f_j|).$$

- A labeled point must be far enough on its known correct side.
- An unlabeled point is encouraged to be far from the boundary on either side.
- This pushes the boundary away from dense regions when the cluster assumption is appropriate.
- The unlabeled term does not itself tell us which side has the correct class meaning.

S3VM encourages a low-density classification boundary



The unlabeled hat loss is nonconvex by a midpoint check

$$h(f) = \max(0, 1 - |f|), \qquad h(-1) = h(1) = 0, \quad h(0) = 1.$$

$$h\left(\frac{-1+1}{2}\right) = 1 > \frac{h(-1) + h(1)}{2} = 0.$$

- A convex function cannot exceed the average endpoint value at their midpoint.
- This explicit violation explains the two-sided preference and the nonconvex S3VM objective.
- Moving away from zero in either direction lowers the unlabeled loss.
- Initialization and approximate search can therefore select different local solutions.
- The supervised convexity guarantee cannot simply be reused after adding this term.

A proportion constraint must reflect evidence rather than force equal classes

An archive is believed to contain about 10% faults

A candidate solution assigns all 1,000 unlabeled rows to the normal class.

- A trusted proportion prior can discourage this all-normal collapse.
- Forcing 500 faults would instead impose a 50% prevalence unsupported by the application.
- The labeled set may have been deliberately balanced, so its ratio is not automatically the deployment ratio.
- Treat prevalence information as another assumption and perform sensitivity checks.
- Even a correct class count does not identify which individual rows are faulty.

Worked S3VM: loss is largest at the boundary

Unlabeled score $f(x)$	-1.4	-0.3	0	0.8
$\max(0, 1 - \lvert f \rvert)$	0	0.7	1	0.2

- At $f = 0$, the point is on the boundary and receives the largest unlabeled penalty.
- Moving to either $f \geq 1$ or $f \leq -1$ removes that penalty.
- For these four scores, the total unlabeled loss is 1.9 before multiplication by C_U.
- The negative and positive sides are equally acceptable to the unlabeled term.
- Check the full objective: reducing this term may worsen labeled hinge loss or the regularizer.

S3VM requires care with optimization and class proportions

- The supervised hinge objective is convex in (w, b); adding $\max(0, 1 - |f|)$ makes the objective nonconvex.
- Different initializations may lead to different local solutions.
- Without other controls, many unlabeled points can be assigned to the same class.
- A class-proportion constraint or prior can reduce collapse, but the correct proportion is not necessarily 50:50.
- Start from a supervised solution, increase unlabeled influence cautiously and select settings with labeled validation.
- Report the optimization rule and assumptions; an approximate result is not a guaranteed global optimum.

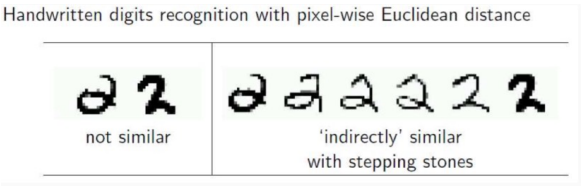
Exercise: three ways to use one unlabeled point

1. Self-training predicts $(0.98, 0.02)$: what hard target is proposed, and what is still unknown?
2. A two-class mixture gives responsibility $(0.98, 0.02)$: how does the point enter the mean updates?
3. S3VM gives score $f = 0.2$: what is the unlabeled loss, and why does it not determine the correct class?
4. Two views confidently disagree: what should be inspected before exchange?

After discussion

Hard target 0 is inferred, not verified; mixture updates use fractional weights; S3VM loss is 0.8; inspect view validity, pairing and calibration.

A graph makes the smoothness assumption explicit



- Nodes are observations; verified labels anchor unknown scores.
- Edges and nonnegative weights encode task-relevant similarity.
- Trusted paths carry label information. The sequence of digit images illustrates similarity through intermediate samples.
- Here a graph expresses neighborhood smoothness. A probabilistic graphical model instead uses a dependency graph to factor a joint probability.

Connection to L11

The same neighborhood structure that supported embedding can support label propagation.

Intermediate observations can connect two distant-looking examples

- Two handwritten 2s may differ greatly under direct pixel distance.
- Intermediate writing styles can form a sequence of locally similar observations.
- A graph lets information travel through these short relations rather than one direct shortcut.
- This helps only when the connecting sequence stays within the relevant class or label-smooth structure.
- A bridge through another digit can spread incorrect labels just as effectively.

Interpretation

The original stepping-stone image is evidence about possible paths, not proof that every nearby image shares a label.

Build weights using a relevant representation and metric

$$W_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right) \quad \text{for selected neighbor edges; otherwise } W_{ij} = 0.$$

At $\sigma = 1$

Distance 1 gives weight $e^{-1/2} \approx 0.607$; distance 3 gives weight $e^{-9/2} \approx 0.011$.

- Choose k-nearest-neighbor or radius edges and make the graph symmetric where required.
- Scale features or use a task-relevant representation before distances are computed.
- Small bandwidth concentrates influence locally; large bandwidth spreads influence more broadly.
- Inspect disconnected components and cross-class edges using available true labels.

Normalize weights to obtain a neighbor-average transition

Node B connects to A and C

Their weights are 1 and 3, and no self-edge is used.

$$D_{BB} = 4, \qquad P_{BA} = 1/4, \quad P_{BC} = 3/4, \qquad P = D^{-1}W.$$

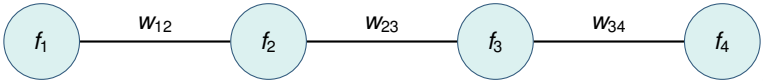
- The row weights sum to one and define a weighted average.
- With scores $f_A = 0$ and $f_C = 1$, one update gives $f_B = 0.25(0) + 0.75(1) = 0.75$.
- P need not be symmetric even when W is symmetric, because node degrees differ.
- Distinguish this random-walk normalization from the symmetric normalization used in label spreading.

Graph energy penalizes different scores across strong edges

$$D_{ii} = \sum_j W_{ij}, \quad L_G = D - W, \quad E(f) = \frac{1}{2} \sum_{i,j} W_{ij}(f_i - f_j)^2 = f^\top L_G f.$$

- f_i is a class score at node i; it is the quantity we want to learn.
- Verified scores are fixed, for example $f_A = 0$ and $f_C = 1$ in a binary teaching example.
- Strong edges make large score differences costly.
- The factor $1/2$ avoids counting an undirected edge twice.
- Without fixed labels or another fidelity term, a constant score gives a trivial zero-energy solution.

Graph energy penalizes different scores across strong edges



$$E(f) = \sum_{(i,j)} w_{ij}(f_i - f_j)^2$$

A three-node Laplacian reproduces the weighted graph energy

$$W = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 3 \\ 0 & 3 & 0 \end{bmatrix}, \quad D = \operatorname{diag}(1, 4, 3), \quad L = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 4 & -3 \\ 0 & -3 & 3 \end{bmatrix}.$$

$$f = (0, b, 1)^\top \quad \Rightarrow \quad f^\top L f = b^2 + 3(b-1)^2.$$

- The middle diagonal is four because its incident weights sum to four.
- With $b = 0.75$, energy is $0.5625 + 0.1875 = 0.75$.
- Row sums of L are zero; off-diagonal negative signs arise from score differences.
- Known endpoint scores are boundary conditions, not variables freely optimized by the graph.

Unknown scores are weighted averages of neighboring scores

$$\frac{\partial E}{\partial f_i} = 0 \quad \Rightarrow \quad f_i = \frac{\sum_j W_{ij} f_j}{\sum_j W_{ij}} \quad (i \in U).$$

$$L_{UU} f_U = -L_{UL} y_L, \qquad f_U = L_{UU}^{-1} W_{UL} y_L \quad \text{when invertible.}$$

- Known labels are held fixed; only the unlabeled block is optimized.
- The first equation is the harmonic or weighted-average condition.
- Each connected component needs a labeled anchor for a uniquely anchored hard-clamped solution.
- Solve a linear system in practice instead of explicitly computing a matrix inverse.

Solve the unlabeled block for the unit-weight four-node chain

A–B–C–D

A has fixed score 0 and D fixed score 1; B and C are unknown.

$$L_{UU} = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \qquad L_{UU} \begin{bmatrix} b \\ c \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

$$2b - c = 0, \quad -b + 2c = 1 \quad \Rightarrow \quad b = 1/3,\ c = 2/3.$$

- The right-hand side collects the influence of the fixed labeled endpoints.
- This is the same solution as the harmonic weighted-average equations.
- Solve the system rather than treating an inverse symbol as an implementation requirement.
- Only the unknown block satisfies stationarity.

Worked propagation: a stronger edge pulls the score closer

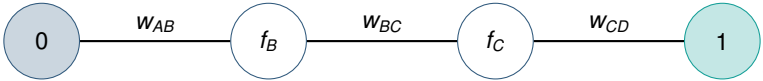
Three nodes

A has verified score 0, C has verified score 1, and B is unlabeled. Edges are $W_{AB} = 1$ and $W_{BC} = 3$.

$$E(f_B) = 1(f_B - 0)^2 + 3(f_B - 1)^2.$$
$$2f_B + 6(f_B - 1) = 0 \quad \Rightarrow \quad f_B = 0.75.$$

- The same result is the weighted average $(1 \times 0 + 3 \times 1)/(1 + 3)$.
- With a threshold of 0.5, B receives class 1 in this scalar example.
- The score is determined by graph relationships and anchors, not by an independent true-label observation of B.
- Equal edge weights would give $f_B = 0.5$.

Worked propagation: a stronger edge pulls the score closer



Unknown scores become weighted averages of their neighbors.

A score can change because the graph changes, even with identical labels

Weights to anchors 0 and 1	Unknown weighted average	Predicted class at threshold 0.5
1 and 3	$(1 \cdot 0 + 3 \cdot 1)/4 = 0.75$	1
3 and 1	$(3 \cdot 0 + 1 \cdot 1)/4 = 0.25$	0
1 and 1	0.5	Tie; needs a stated rule

- True anchor labels have not changed; only similarity strengths changed.
- Parameter sensitivity is model sensitivity, not automatically numerical failure.
- Inspect whether the chosen representation justifies stronger influence from one anchor.

Worked iteration: two unknown nodes settle between fixed anchors

Unit-weight chain A–B–C–D

A is fixed at 0, D at 1. Initialize B=C=0.5 and update both synchronously.

Iteration	A	B	C	D
0	0	0.500	0.500	1
1	0	0.250	0.750	1
2	0	0.375	0.625	1
3	0	0.3125	0.6875	1

$$f_B = f_C/2, \quad f_C = (f_B + 1)/2 \quad \Rightarrow \quad f_B = 1/3,\ f_C = 2/3.$$

Clamping

Restore A and D to their verified values after every update.

Synchronous and sequential averaging have different intermediate states

Four-node chain, initial $b = c = 0.5$

Endpoints remain 0 and 1.

Update rule	First new B	First new C
Synchronous old-state update	$0.5/2 = 0.25$	$(0.5 + 1)/2 = 0.75$
Sequential B then C	$0.5/2 = 0.25$	$(0.25 + 1)/2 = 0.625$

- The sequential rule uses the newly updated B immediately.
- Under the anchored convergent setup, both seek the same harmonic solution $(1/3, 2/3)$.
- State update order, convergence criterion and clamping behavior when comparing iteration traces.

Multiclass propagation uses one score column per class

$$F \in \mathbb{R}^{(L+U)\times C}, \quad Y_i = (0,\ldots,1,\ldots,0) \text{ for a verified class}, \quad \hat{y}_i = \arg\max_c F_{ic}.$$

- Apply neighbor averaging to each score column, keeping verified rows fixed.
- The graph supplies the same structural relations for every class.
- With neighboring one-hot labels $(1,0,0)$ and $(0,1,0)$ and edge weights 1 and 3, the unknown row becomes $(0.25, 0.75, 0)$.
- Row-normalized harmonic scores can be interpreted under the chosen propagation model; do not assume perfect probabilistic calibration.
- Use sparse matrices when graphs are large.

Each multiclass row combines neighbors but does not verify a label

Three classes

Neighbor A has verified vector $(1, 0, 0)$; neighbor C has $(0, 1, 0)$; their weights are 1 and 3.

$$F_B = \frac{1(1, 0, 0) + 3(0, 1, 0)}{4} = (0.25, 0.75, 0).$$

- The largest entry selects class 2 under one-based class naming.
- The zero third entry means no class-3 anchor influence reaches B in this tiny example.
- The row sums to one because weighted averaging preserves this property under these boundary conditions.
- Those values remain graph-derived scores; independent calibration is a separate question.
- A real class absent from the anchors cannot be named merely by increasing propagation iterations.

Label spreading balances smoothness and label fidelity

$$S = D^{-1/2} W D^{-1/2}, \qquad F^{(t+1)} = \alpha S F^{(t)} + (1-\alpha) Y, \quad 0 < \alpha < 1.$$
$$F^{*} = (1-\alpha)(I - \alpha S)^{-1} Y.$$

- Each step propagates through the normalized graph and restores some influence from initial labels.
- Unlike hard clamping, this form can change scores at labeled nodes while penalizing departure from their targets.
- Large α gives stronger propagation; small α stays closer to initial label information.
- The zero rows for unlabeled data are an initialization, not verified membership in a zero-valued class.

A two-node label-spreading example shows soft fidelity

$$S = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad Y = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \alpha = 0.8.$$

$$F^{(1)} = 0.8SY + 0.2Y = (0.2, 0.8)^\top, \qquad F^* = (5/9, 4/9)^\top.$$

- The labeled node's score is not hard-clamped to one in this soft scheme.
- At equilibrium, $a = 0.8b + 0.2$ and $b = 0.8a$, giving $a \approx 0.556, b \approx 0.444$.
- This is one class-score column, not two class probabilities for one observation.
- If class probabilities are desired, row normalization must follow the actual multiclass model.

A graph can spread errors as efficiently as labels

Problem	Consequence	Targeted check
Component without any labeled anchor	No uniquely anchored hard-propagation labels	Count components and label coverage
Strong cross-class edges	Wrong labels travel across the boundary	Inspect known-label edges and representative neighbors
Representation dominated by background	Smooth but irrelevant predictions	Examine original samples and task features
Highly unbalanced graph density	Unequal influence across regions	Compare normalization and class-wise validation

Response

Fix or qualify the graph assumption before adding more propagation iterations.

An unanchored component has no unique absolute label score

Two connected unlabeled nodes and no labeled path

Their energy is $(f_1 - f_2)^2$.

$$f_1 = f_2 = c \quad \Rightarrow \quad E = 0 \quad \text{for any constant } c.$$

- The graph determines agreement, but not which class the shared score represents.
- Its Laplacian block is singular because the constant mode has zero energy.
- Add a justified labeled anchor, handle the component separately or explicitly specify another prior.
- Replacing a missing anchor with an arbitrary default class silently adds unsupported information.
- Invertibility and label coverage are substantive learning conditions, not only implementation details.

Partial supervision can specify relationships without class names

- **Seed label:** a few observations have known cluster or class identities.
- **Must-link** (i, j): the two observations must be assigned to the same cluster.
- **Cannot-link** (i, j): the two observations must be assigned to different clusters.
- Pairwise constraints do not necessarily specify the name or total number of clusters.
- Constrained clustering balances compact groups with the supplied domain knowledge.

Customer records

Two records confirmed to describe the same customer should stay together; records known to represent distinct identities should not merge in an identity-clustering task.

Task matters

Different identities need not be different clusters for every other clustering objective.

Seeded clustering uses known groups to initialize the centers

Scalar observations

Seeded group A has $-1, 1$; group B has $9, 11$. Unlabeled values are 2 and 8.

$$\mu_A = 0, \quad \mu_B = 10 \quad \Rightarrow \quad 2 \to A,\ 8 \to B.$$
$$\mu_A^{\text{new}} = (-1 + 1 + 2)/3 = 2/3, \quad \mu_B^{\text{new}} = (9 + 11 + 8)/3 = 28/3.$$

- Seed identities provide partial group meaning; unlabeled rows affect the fitted centers.
- State whether verified seed assignments remain fixed or only initialize a later free clustering.
- This is distinct from knowing only pairwise must-link relationships.

Must-link is transitive; cannot-link is not

Constraint logic

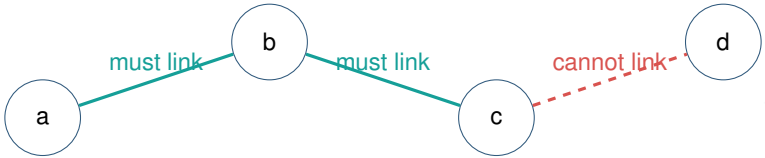
If A must-link B and B must-link C, then A, B and C must belong to one connected must-link component.

- A cannot-link C inside that component creates an immediate contradiction.
- A cannot-link B and B cannot-link C do **not** imply A cannot-link C; A and C may share a cluster.
- Collapse must-link components conceptually before checking cannot-link relations.
- The remaining cannot-link graph may still be infeasible for the chosen number of clusters.

Two-cluster limit

Three groups with pairwise cannot-link constraints require at least three clusters, even if no individual pair contradicts must-links.

Must-link is transitive; cannot-link is not



The must-link component is indivisible; exclusions are checked between components.

Collapse must-link components before checking exclusion constraints

Two known relationships

A must-link B and B must-link C, so define component $G_1 = \{A, B, C\}$. A separate point D forms G_2.

- If C cannot-link D, the two components must receive different cluster assignments.
- If A cannot-link C, the contradiction lies inside G_1 and cannot be repaired by adding clusters.
- Three components mutually excluding one another require at least three clusters.
- This pre-check simplifies the problem but does not guarantee that a greedy assignment will find every feasible solution.
- Do not take a transitive closure of cannot-link as if it were must-link.

Worked constrained assignment: the nearest cluster may be forbidden

Two centers on a line

Centers are $\mu_1 = 0$ and $\mu_2 = 5$. Point A at $x_A = 0$ is assigned to cluster 1. Point B is at $x_B = 1$ and cannot-link A.

Candidate cluster for B	Squared distance	Constraint satisfied?	Decision
1	$(1-0)^2 = 1$	No	Forbidden
2	$(1-5)^2 = 16$	Yes	Assign B here

- A hard constrained assignment selects the nearest **feasible** cluster.
- If no cluster is feasible, report failure rather than silently violating the constraint.
- Assignment order and initialization can matter in greedy constrained K-means variants.

Soft penalties and hard constraints define different clustering problems

Point B at 1, centers 0 and 5

B cannot-link A, which is already at center 0.

Assignment	Squared distance	Soft cost with violation penalty ρ
First cluster	1	$1 + \rho$
Second cluster	16	16

- With $\rho = 10$, a soft objective prefers cost 11 and violates the pair constraint.
- With $\rho = 20$, it prefers the feasible second cluster at cost 16.
- A hard constraint forbids the first cluster regardless of cost.
- Report violations instead of describing a soft optimum as fully feasible.

Exercise: check constraints before clustering

1. A must-link B; B must-link C; A cannot-link C. Can any number of clusters satisfy this set?
2. A cannot-link B; B cannot-link C. Can A and C share a cluster?
3. Three must-link components have pairwise cannot-link relations. Are two clusters sufficient?
4. A greedy constrained assignment fails for one ordering. Does that alone prove the entire problem is infeasible?

After discussion

The first set contradicts itself; A and C may share; three pairwise-excluding components need at least three clusters; one greedy failure is not a proof of global infeasibility.

Distinguish contradiction, insufficient clusters and a failed search

Finding	What it establishes	Appropriate response
Must-link and cannot-link conflict within one component	Logical infeasibility	Recheck constraint source
Three mutually excluded groups, but $K = 2$	Too few clusters for these constraints	Reconsider K or the constraints
One greedy ordering fails	That search found no feasible assignment	Try justified alternative search; do not claim a global proof

- Keep the original constraints visible while diagnosing the cause.
- Silently dropping a difficult pair changes the question being solved.

Evaluate gains at a fixed real-label budget

Comparison	Keep the same	Report
Supervised vs semi-supervised	True labeled training set, test labels, capacity where possible	Test score change and repeated-split variability
Different unlabeled methods	Available unlabeled pool and inductive/transductive setting	Class-wise scores and computational cost
Different label budgets	A clearly stated sampling rule	Learning curve versus number of real labels

- Never use validation or test true labels as pseudo-label targets.
- State input-distribution shift, unlabeled contamination and per-class coverage.
- Compare annotation cost as well as performance when deployment is the goal.

Report a learning curve against the true-label budget

An experimental plan, not measured results

Compare budgets of 10, 20 and 40 training labels using repeated samples from the same labeled pool.

- At each budget, compare supervised and semi-supervised predictors on the same independent validation/test protocol.
- Keep unlabeled availability and inductive or transductive access rules fixed.
- Record mean score, variation across splits, per-class recall and computation cost.
- A benefit at 10 labels may disappear at 40; report that dependence rather than one favorable run.
- Distinguish training annotation cost from labels used to establish the evaluation benchmark.

Match the method to the structural evidence

Available evidence	Candidate	Main risk
Reliable confidence	Self-training	Confident mistakes reinforce themselves
Complementary views	Co-training / consistency	Shared errors; invalid transformations
Plausible class density	Generative model	Wrong distribution; local optimum
Low-density gaps	S3VM	Nonconvex fit; class collapse
Trusted similarity graph	Label propagation / spreading	Cross-class edges; missing anchors
Reliable pair constraints	Constrained clustering	Contradictory or irrelevant constraints

Always

Use labeled validation and a supervised baseline.

Exit exercise: design a safe experiment for rare faults

Sensor archive

You have 40 labeled training records, 5,000 unlabeled records, and separate labeled validation/test sets. A rare fault is underrepresented; records from another device may be present.

1. State a supervised baseline and a plausible structural assumption.
2. Propose one unlabeled method and calculate or describe its first update.
3. Explain how you will detect rare-class collapse or out-of-domain contamination.
4. Give one reason to stop using unlabeled data even if training confidence rises.

A complete answer

Input boundary, true-label budget, structural evidence, numerical update, independent validation and a failure criterion.